```
    username=current_user.username,
        action="logout",
        resource_type="auth",
        description=f"用户 {current_user.username} 登出",
        request=request
    )

    return ResponseBase(message="登出成功")

@router.post("/password/reset", response_model=ResponseBase, summary="请求密码重置")
async def request_password_reset(
    reset_request: PasswordResetRequest,
    db: AsyncSession = Depends(get_db)
):
    """
    请求密码重置

    - 发送重置链接到注册邮箱
    - 实际生产环境需配置 SMTP
    """
    # 查找用户
    result = await db.execute(select(User).where(User.email == reset_request.email))
    user = result.scalar_one_or_none()

    # 即使用户不存在也返回成功，防止邮箱枚举
    if user:
        reset_token = create_password_reset_token(user.email)
        # TODO: 发送邮件
        # 这里仅打印 token 用于测试
        print(f"Password reset token for {user.email}: {reset_token}")

    return ResponseBase(message="如果邮箱存在，重置链接已发送")

@router.post("/password/reset/confirm", response_model=ResponseBase, summary="确认密码重置")
async def confirm_password_reset(
    confirm_data: PasswordResetConfirm,
    db: AsyncSession = Depends(get_db)
):
    """
    确认密码重置

    - 验证重置令牌
    - 设置新密码
    """
    email = verify_password_reset_token(confirm_data.token)
    if not email:
        raise InvalidTokenError("无效或过期的重置令牌")

    # 查找用户
    result = await db.execute(select(User).where(User.email == email))
    user = result.scalar_one_or_none()

```

```
    if not user:
        raise UserNotFoundError(email)

    # 更新密码
    user.hashed_password = get_password_hash(confirm_data.new_password)
    await db.commit()

    return ResponseBase(message="密码重置成功")

@router.post("/password/change", response_model=ResponseBase, summary="修改密码
")
async def change_password(
    request: Request,
    change_data: ChangePasswordRequest,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    修改当前用户密码

    - 需要验证旧密码
    """
    if not verify_password(change_data.old_password,
current_user.hashed_password):
        raise InvalidCredentialsError("旧密码错误")

    current_user.hashed_password = get_password_hash(change_data.new_password)
    await db.commit()

    # 记录审计日志
    await AuditService.log(
        db=db,
        user_id=current_user.id,
        username=current_user.username,
        action="change_password",
        resource_type="user",
        resource_id=str(current_user.id),
        description=f"用户 {current_user.username} 修改密码",
        request=request
    )

    return ResponseBase(message="密码修改成功")

@router.get("/me", response_model=DataResponse[UserInfo], summary="获取当前用户
信息")
async def get_current_user_info(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取当前登录用户的详细信息

    - 包含角色和权限信息
    """
    # 获取用户权限
```

```
    permissions = []
    if current_user.role:
        for rp in current_user.role.role_permissions:
            permissions.append(rp.permission.code)

    user_info = UserInfo(
        id=str(current_user.id),
        username=current_user.username,
        email=current_user.email,
        full_name=current_user.full_name,
        phone=current_user.phone,
        avatar_url=current_user.avatar_url,
        role_name=current_user.role.name if current_user.role else None,
        role_display_name=current_user.role.display_name if current_user.role
else None,
        organization_name=current_user.organization.name if
current_user.organization else None,
        permissions=permissions,
        is_active=current_user.is_active,
        created_at=current_user.created_at,
        last_login_at=current_user.last_login_at
    )

    result = await db.execute(select(Device).where(Device.id == device_id))
    device = result.scalar_one_or_none()

    if not device:
        raise DeviceNotFoundError(device_id)

    device_name = device.name
    await db.delete(device)
    await db.commit()

    await AuditService.log(
        db=db,
        user_id=current_user.id,
        username=current_user.username,
        action="delete",
        resource_type="device",
        resource_id=device_id,
        description=f"删除设备 {device_name}",
        request=request
    )

    return ResponseBase(message="设备删除成功")

# ========== 批次管理 ==========

@router.get("/batches", response_model=PaginatedResponse[BatchResponse],
summary="获取批次列表")
async def list_batches(
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=20, ge=1, le=100),
    search: Optional[str] = None,
    production_line_id: Optional[str] = None,
    status: Optional[str] = None,
```

```
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取批次列表
    """
    query = select(Batch).options(selectinload(Batch.production_line))
    count_query = select(func.count(Batch.id))

    if search:
        search_filter = or_(
            Batch.batch_no.ilike(f"%{search}%"),
            Batch.name.ilike(f"%{search}%")
        )
        query = query.where(search_filter)
        count_query = count_query.where(search_filter)

    if production_line_id:
        query = query.where(Batch.production_line_id == production_line_id)
        count_query = count_query.where(Batch.production_line_id ==
production_line_id)

    if status:
        query = query.where(Batch.status == status)
        count_query = count_query.where(Batch.status == status)

    total_result = await db.execute(count_query)
    total = total_result.scalar()

    query = query.order_by(Batch.created_at.desc())
    query = query.offset((page - 1) * page_size).limit(page_size)

    result = await db.execute(query)
    batches = result.scalars().all()

    batch_list = [
        BatchResponse(
            id=str(b.id),
            batch_no=b.batch_no,
            name=b.name,
            product_type=b.product_type,


    while current_time <= sim_request.end_time:
        # 基础值 + 趋势 + 噪声
        value = sim_request.base_value + \
                sim_request.trend_slope * step + \
                generate_normal_random(0, sim_request.noise_std)

        # 随机添加异常
        if generate_random_value() < sim_request.anomaly_ratio:
            # 异常值：偏离 3-5 个标准差
            anomaly_factor = random_select([-1, 1]) *
generate_uniform_random(3, 5)
            value += anomaly_factor * sim_request.noise_std

```

```
223             point = TimeseriesPoint(
224                 id=uuid.uuid4(),
225                 dataset_id=dataset.id,
226                 timestamp=current_time,
227                 value=value,
228                 quality=100
229             )
230             points.append(point)
231             values.append(value)
232             timestamps.append(current_time)
233 
234             current_time += timedelta(seconds=sim_request.frequency_seconds)
235             step += 1
236 
237         if points:
238             db.add_all(points)
239 
240             # 更新统计信息
241             dataset.total_points = len(points)
242             dataset.start_time = min(timestamps)
243             dataset.end_time = max(timestamps)
244             extra = dataset.extra_data or {}
245             extra.update({
246                 "min_value": float(min(values)),
247                 "max_value": float(max(values)),
248                 "mean_value": float(sum(values) / len(values)),
249                 "std_value": float(calculate_std(values))
250             })
251             dataset.extra_data = extra
252 
253             await db.commit()
254 
255         await db.refresh(dataset)
256 
257         extra = dataset.extra_data or {}
258 
259         return DataResponse(
260             data=TimeseriesDatasetResponse(
261                 id=str(dataset.id),
262                 name=dataset.name,
263                 description=dataset.description,
264                 source_type=dataset.source_type,
265                 device_id=str(dataset.device_id) if dataset.device_id else None,
266                 sensor_name=extra.get("sensor_name"),
267                 parameter_name=extra.get("parameter_name"),
268                 unit=extra.get("unit"),
269                 start_time=dataset.start_time,
270                 end_time=dataset.end_time,
271                 point_count=dataset.total_points or 0,
272                 min_value=extra.get("min_value"),
273                 max_value=extra.get("max_value"),
274                 mean_value=extra.get("mean_value"),
275                 std_value=extra.get("std_value"),
276                 status=dataset.status,
277                 created_at=dataset.created_at
278             ),
```

```
        message=f"模拟数据生成成功，共 {len(points)} 个数据点"
    )

@router.get("/datasets",
response_model=PaginatedResponse[TimeseriesDatasetListResponse], summary="获取
数据集列表")
async def list_timeseries_datasets(
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=20, ge=1, le=100),
    search: Optional[str] = None,
    device_id: Optional[str] = None,
    source_type: Optional[str] = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取时序数据集列表
    """
    query =
select(TimeseriesDataset).options(selectinload(TimeseriesDataset.device))
    count_query = select(func.count(TimeseriesDataset.id))

    if search:
        search_filter = or_(
            TimeseriesDataset.name.ilike(f"%{search}%"),
            TimeseriesDataset.description.ilike(f"%{search}%")
        )
        query = query.where(search_filter)
        count_query = count_query.where(search_filter)

    if device_id:
        query = query.where(TimeseriesDataset.device_id == device_id)
        count_query = count_query.where(TimeseriesDataset.device_id ==
device_id)

    if source_type:
        query = query.where(TimeseriesDataset.source_type == source_type)
        count_query = count_query.where(TimeseriesDataset.source_type ==
source_type)

    total_result = await db.execute(count_query)
    total = total_result.scalar()

    query = query.order_by(TimeseriesDataset.created_at.desc())
    query = query.offset((page - 1) * page_size).limit(page_size)

    result = await db.execute(query)
    datasets = result.scalars().all()

    dataset_list = [
        TimeseriesDatasetListResponse(
            id=str(d.id),
            name=d.name,
            device_name=d.device.name if d.device else None,
            sensor_name=d.extra_data.get('sensor_name') if d.extra_data else
None,
```

```
            point_count=d.total_points or 0,
            start_time=d.start_time,
            end_time=d.end_time,
            status=d.status,
            created_at=d.created_at
        )
        for d in datasets
    ]

    total_pages = (total + page_size - 1) // page_size

    return PaginatedResponse(
        data=dataset_list,
        meta=PaginationMeta(
            total=total,
            page=page,
            page_size=page_size,
            total_pages=total_pages,
            has_next=page < total_pages,
            has_prev=page > 1
        )
    )

@router.get("/datasets/{dataset_id}",
response_model=DataResponse[TimeseriesDatasetResponse], summary="获取数据集详情
")
async def get_timeseries_dataset(
    dataset_id: str,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取时序数据集详情
    """
    result = await db.execute(
        select(TimeseriesDataset)
        .options(selectinload(TimeseriesDataset.device))
        .where(TimeseriesDataset.id == dataset_id)
    )
    dataset = result.scalar_one_or_none()

    if not dataset:
        raise ResourceNotFoundError("时序数据集", dataset_id)

    extra = dataset.extra_data or {}

    return DataResponse(
        data=TimeseriesDatasetResponse(
            id=str(dataset.id),
            name=dataset.name,
            description=dataset.description,
            source_type=dataset.source_type,
            file_path=dataset.file_path,
            file_name=extra.get("file_name"),
            device_id=str(dataset.device_id) if dataset.device_id else None,
            device_name=dataset.device.name if dataset.device else None,
```

```
            sensor_name=extra.get("sensor_name"),
            parameter_name=extra.get("parameter_name"),
            unit=extra.get("unit"),
            start_time=dataset.start_time,
            end_time=dataset.end_time,
            point_count=dataset.total_points or 0,
            min_value=extra.get("min_value"),
            max_value=extra.get("max_value"),
            mean_value=extra.get("mean_value"),
            std_value=extra.get("std_value"),
            status=dataset.status,
            uploaded_by=str(dataset.created_by) if dataset.created_by else
None,
            created_at=dataset.created_at
        )
    )

@router.get("/datasets/{dataset_id}/data",
response_model=DataResponse[TimeseriesDataResponse], summary="获取时序数据")
async def get_timeseries_data(
    dataset_id: str,
    start_time: Optional[datetime] = None,
    end_time: Optional[datetime] = None,
    limit: int = Query(default=1000, le=10000),
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取时序数据点（用于绘图）

    - 支持时间范围筛选
    - 默认返回最多 1000 个点
    """
    # 验证数据集存在
    result = await
db.execute(select(TimeseriesDataset).where(TimeseriesDataset.id == dataset_id))
    dataset = result.scalar_one_or_none()

    if not dataset:
        raise ResourceNotFoundError("时序数据集", dataset_id)

    # 查询数据点
    query = select(TimeseriesPoint).where(TimeseriesPoint.dataset_id ==
dataset_id)

    if start_time:
        query = query.where(TimeseriesPoint.timestamp >= start_time)
    if end_time:
        query = query.where(TimeseriesPoint.timestamp <= end_time)

    query = query.order_by(TimeseriesPoint.timestamp).limit(limit)

    result = await db.execute(query)
    points = result.scalars().all()

    point_responses = [
```

```
        TimeseriesPointResponse(
            timestamp=p.timestamp,
            value=p.value,
            quality=p.quality
        )
        for p in points
    ]

    # 计算统计信息
    if points:
        values = [p.value for p in points]
        statistics = {
            "count": len(values),
            "min": min(values),
            "max": max(values),
            "mean": sum(values) / len(values),
            "std": calculate_std(values)
        }
    else:
        statistics = None

    return DataResponse(
        data=TimeseriesDataResponse(
            dataset_id=str(dataset.id),
            dataset_name=dataset.name,
            unit=(dataset.extra_data or {}).get("unit"),
            points=point_responses,
            statistics=statistics
        )
    )

@router.post("/analyze", response_model=DataResponse[TriggerAnalysisResponse],
summary="触发时序分析")
async def trigger_timeseries_analysis(
    request: Request,
    analysis_request: TimeseriesAnalysisRequest,
    background_tasks: BackgroundTasks,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    触发时序分析任务

    - 支持统计分析、异常检测、趋势分析
    - 异步执行
    """
    # 验证数据集存在
    result = await db.execute(
        select(TimeseriesDataset).where(TimeseriesDataset.id ==
analysis_request.dataset_id)
    )
    dataset = result.scalar_one_or_none()

    if not dataset:
        raise ResourceNotFoundError("时序数据集", analysis_request.dataset_id)

```

```
    # 创建分析任务
    job = AnalysisJob(
        id=uuid.uuid4(),
        name=f"时序分析-{dataset.name}-
{datetime.now().strftime('%Y%m%d%H%M%S')}",
        job_type="anomaly_detection",
        description=f"分析数据集: {dataset.name}",
        status="pending",
        input_params={
            "dataset_id": analysis_request.dataset_id,
            "start_time": analysis_request.start_time.isoformat() if
analysis_request.start_time else None,
            "end_time": analysis_request.end_time.isoformat() if
analysis_request.end_time else None,
            "analysis_types": analysis_request.analysis_types,
            "anomaly_method": analysis_request.anomaly_method,
            "anomaly_threshold": analysis_request.anomaly_threshold
        },
        dataset_id=uuid.UUID(analysis_request.dataset_id),
        created_by=current_user.id
    )

    db.add(job)
    await db.commit()
    await db.refresh(job)

    # 在后台执行分析
    background_tasks.add_task(
        timeseries_service.run_analysis,
        str(job.id),
        analysis_request.dataset_id,
        analysis_request.anomaly_method,
        analysis_request.anomaly_threshold,
        analysis_request.start_time,
        analysis_request.end_time
    )

    await AuditService.log(
        db=db,
        user_id=current_user.id,
        username=current_user.username,
        action="trigger_analysis",
        resource_type="timeseries_analysis",
        resource_id=str(job.id),
        description=f"触发时序分析任务: {dataset.name}",
        request=request
    )

    return DataResponse(
        data=TriggerAnalysisResponse(
            job_id=str(job.id),
            dataset_id=analysis_request.dataset_id,
            message="分析任务已创建"
        )
    )

```

```
@router.get("/anomalies",
response_model=PaginatedResponse[TimeseriesAnomalyResponse], summary="获取异常
点列表")
async def list_timeseries_anomalies(
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=20, ge=1, le=100),
    dataset_id: Optional[str] = None,
    anomaly_type: Optional[str] = None,
    severity: Optional[str] = None,
    is_confirmed: Optional[bool] = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取异常点列表
    """
    query = select(TimeseriesAnomaly)
    count_query = select(func.count(TimeseriesAnomaly.id))

    if dataset_id:
        query = query.where(TimeseriesAnomaly.dataset_id == dataset_id)
        count_query = count_query.where(TimeseriesAnomaly.dataset_id ==
dataset_id)

    if anomaly_type:
        query = query.where(TimeseriesAnomaly.anomaly_type == anomaly_type)
        count_query = count_query.where(TimeseriesAnomaly.anomaly_type ==
anomaly_type)

    if severity:
        query = query.where(TimeseriesAnomaly.severity == severity)
        count_query = count_query.where(TimeseriesAnomaly.severity == severity)

    if is_confirmed is not None:
        query = query.where(TimeseriesAnomaly.is_confirmed == is_confirmed)
        count_query = count_query.where(TimeseriesAnomaly.is_confirmed ==
is_confirmed)

    total_result = await db.execute(count_query)
    total = total_result.scalar()

    query = query.order_by(TimeseriesAnomaly.timestamp.desc())
    query = query.offset((page - 1) * page_size).limit(page_size)

    result = await db.execute(query)
    anomalies = result.scalars().all()

    anomaly_list = [
        TimeseriesAnomalyResponse(
            id=str(a.id),
            dataset_id=str(a.dataset_id),
            timestamp=a.timestamp,
            value=a.value,
            anomaly_type=a.anomaly_type,
            severity=a.severity,
            score=a.score,
```

```
            detection_method=a.detection_method,
            threshold=a.threshold,
            description=a.description,
            is_confirmed=a.is_confirmed,
            is_false_positive=a.is_false_positive,
            created_at=a.created_at
        )
        for a in anomalies
    ]

    total_pages = (total + page_size - 1) // page_size

    return PaginatedResponse(
        data=anomaly_list,
        meta=PaginationMeta(
            total=total,
            page=page,
            page_size=page_size,
            total_pages=total_pages,
            has_next=page < total_pages,
            has_prev=page > 1
        )
    )

@router.get("/reports",
response_model=PaginatedResponse[AnalysisReportResponse], summary="获取分析报告
列表")
async def list_analysis_reports(
    page: int = Query(default=1, ge=1),
    page_size: int = Query(default=20, ge=1, le=100),
    report_type: Optional[str] = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取分析报告列表
    """
    query = select(AnalysisReport)
    count_query = select(func.count(AnalysisReport.id))

    if report_type:
        query = query.where(AnalysisReport.report_type == report_type)
        count_query = count_query.where(AnalysisReport.report_type ==
report_type)

    total_result = await db.execute(count_query)
    total = total_result.scalar()

    query = query.order_by(AnalysisReport.created_at.desc())
    query = query.offset((page - 1) * page_size).limit(page_size)

    result = await db.execute(query)
    reports = result.scalars().all()

    report_list = [
        AnalysisReportResponse(
```

```
            id=str(r.id),
            title=r.title,
            report_type=r.report_type,
            description=r.description,
            content_json=r.content_json,
            content_markdown=r.content_markdown,
            dataset_id=str(r.dataset_id) if r.dataset_id else None,
            device_id=str(r.device_id) if r.device_id else None,
            job_id=str(r.job_id) if r.job_id else None,
            export_path=r.export_path,
            created_by=str(r.created_by) if r.created_by else None,
            created_at=r.created_at
        )
        for r in reports
    ]

    total_pages = (total + page_size - 1) // page_size

    return PaginatedResponse(
        data=report_list,
        meta=PaginationMeta(
            total=total,
            page=page,
            page_size=page_size,
            total_pages=total_pages,
            has_next=page < total_pages,
            has_prev=page > 1
        )
    )

@router.get("/reports/{report_id}",
response_model=DataResponse[AnalysisReportResponse], summary="获取报告详情")
async def get_analysis_report(
    report_id: str,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    获取分析报告详情
    """
    result = await db.execute(select(AnalysisReport).where(AnalysisReport.id
== report_id))
    report = result.scalar_one_or_none()

    if not report:
        raise ResourceNotFoundError("分析报告", report_id)

    return DataResponse(
        data=AnalysisReportResponse(
            id=str(report.id),
            title=report.title,
            report_type=report.report_type,
            description=report.description,
            content_json=report.content_json,
            content_markdown=report.content_markdown,
            dataset_id=str(report.dataset_id) if report.dataset_id else None,
```

```
            device_id=str(report.device_id) if report.device_id else None,
            job_id=str(report.job_id) if report.job_id else None,
            export_path=report.export_path,
            created_by=str(report.created_by) if report.created_by else None,
            created_at=report.created_at
        )
    )

@router.delete("/datasets/{dataset_id}", response_model=ResponseBase,
summary="删除数据集")
async def delete_timeseries_dataset(
    request: Request,
    dataset_id: str,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db)
):
    """
    删除时序数据集
    """
    result = await
db.execute(select(TimeseriesDataset).where(TimeseriesDataset.id == dataset_id))
    dataset = result.scalar_one_or_none()

    if not dataset:
        raise ResourceNotFoundError("时序数据集", dataset_id)

    dataset_name = dataset.name

    # 删除文件
    if dataset.file_path and os.path.exists(dataset.file_path):
        os.remove(dataset.file_path)

    await db.delete(dataset)
    await db.commit()

    await AuditService.log(
        db=db,
        user_id=current_user.id,
        username=current_user.username,
        action="delete",
        resource_type="timeseries_dataset",
        resource_id=dataset_id,
        description=f"删除时序数据集 {dataset_name}",
        request=request
    )

    return ResponseBase(message="数据集删除成功")
"""
Users API Endpoints
用户管理 API 端点
"""
# 以下引入平台核心依赖模块（数据库、路由、校验、ORM 等）
from typing import Optional, List
from datetime import datetime
import uuid

```

```
783  from app.core.database import get_db
784  from app.core.security import (
785      get_password_hash,
786      get_current_user,
787      require_admin,
788      require_superuser,
789      PermissionChecker
790  )
791  from app.core.config import settings
792  from app.core.exceptions import (
793      UserNotFoundError,
794      UsernameAlreadyExistsError,
795      EmailAlreadyExistsError,
796      RoleNotFoundError,
797      PermissionDeniedError,
798      ResourceNotFoundError
799  )
800  from app.models.user import User, Organization
801  from app.models.rbac import Role, Permission, RolePermission
802  from app.schemas.user import (
803      UserCreate,
804      UserUpdate,
805      UserResponse,
806      UserListResponse,
807      UserResetPassword,
808      AssignRoleRequest,
809      RoleCreate,
810      RoleUpdate,
811      RoleResponse,
812      PermissionResponse,
813      OrganizationCreate,
814      OrganizationUpdate,
815      OrganizationResponse
816  )
817  from app.schemas.common import (
818      ResponseBase,
819      DataResponse,
820      PaginatedResponse,
821      PaginationMeta
822  )
823  from app.services.audit_service import AuditService
824
825  router = APIRouter()
826
827  # ========== 用户管理 ==========
828
829  @router.get("", response_model=PaginatedResponse[UserListResponse], summary="
830  获取用户列表")
831  async def list_users(
832      page: int = Query(default=1, ge=1),
833      page_size: int = Query(default=20, ge=1, le=100),
834      search: Optional[str] = None,
835      role_id: Optional[str] = None,
836      is_active: Optional[bool] = None,
837      current_user: User = Depends(require_admin),
838      db: AsyncSession = Depends(get_db)
```

```
839 ):
840     """
841     获取用户列表（需要管理员权限）
842 
843     - 支持搜索用户名、邮箱、姓名
844     - 支持按角色、状态筛选
845     """
846     query = select(User).options(selectinload(User.role),
847 selectinload(User.organization))
848     count_query = select(func.count(User.id))
849 
850     # 搜索过滤
851     if search:
852         search_filter = or_(
853             User.username.ilike(f"%{search}%"),
854             User.email.ilike(f"%{search}%"),
855             User.full_name.ilike(f"%{search}%")
856         )
857         query = query.where(search_filter)
858         count_query = count_query.where(search_filter)
859 
860     if role_id:
861         query = query.where(User.role_id == role_id)
862         count_query = count_query.where(User.role_id == role_id)
863 
864     if is_active is not None:
865         query = query.where(User.is_active == is_active)
866         count_query = count_query.where(User.is_active == is_active)
867 
868     # 非超级用户只能查看同组织用户
869     if current_user.role and current_user.role.name != "superuser":
870         if current_user.organization_id:
871             query = query.where(User.organization_id ==
872 current_user.organization_id)
873             count_query = count_query.where(User.organization_id ==
874 current_user.organization_id)
875 
876     # 分页
877     total_result = await db.execute(count_query)
878     total = total_result.scalar()
879 
880     query = query.order_by(User.created_at.desc())
881     query = query.offset((page - 1) * page_size).limit(page_size)
882 
883     result = await db.execute(query)
884     users = result.scalars().all()
885 
886     # 构建响应
887     user_list = []
888     for user in users:
889         user_list.append(UserListResponse(
890             id=str(user.id),
891             username=user.username,
892             email=user.email,
893             full_name=user.full_name,
894             is_active=user.is_active,
```

```
            role_name=user.role.display_name if user.role else None,
            organization_name=user.organization.name if user.organization else
None,
            created_at=user.created_at,
            last_login_at=user.last_login_at
        ))

    total_pages = (total + page_size - 1) // page_size

    return PaginatedResponse(
        data=user_list,
        meta=PaginationMeta(
            total=total,
            page=page,
            page_size=page_size,
            total_pages=total_pages,
            has_next=page < total_pages,
            has_prev=page > 1
        )
    )

@router.post("", response_model=DataResponse[UserResponse], summary="创建用户")
async def create_user(
    request: Request,
# -*- coding: utf-8 -*-
"""
分析任务调度器
"""
import uuid
import threading
import queue
from datetime import datetime, timedelta
from typing import Dict, List, Optional, Callable
from enum import Enum


class TaskPriority(Enum):
    LOW = 1
    NORMAL = 5
    HIGH = 8
    URGENT = 10


class TaskState(Enum):
    QUEUED = "queued"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"
    TIMEOUT = "timeout"


class AnalysisTask:
    """分析任务对象"""

    def __init__(self, name: str, task_type: str, params: Dict = None,
```

```
                 priority: TaskPriority = TaskPriority.NORMAL,
                 timeout_seconds: int = 3600, created_by: str = None):
        self.task_id = str(uuid.uuid4())
        self.name = name
        self.task_type = task_type
        self.params = params or {}
        self.priority = priority
        self.timeout_seconds = timeout_seconds
        self.created_by = created_by

        self.state = TaskState.QUEUED
        self.progress = 0.0
        self.result = None
        self.error_message = None

        self.created_at = datetime.utcnow()
        self.started_at = None
        self.completed_at = None

    def to_dict(self) -> Dict:
        elapsed = None
        if self.started_at:
            end = self.completed_at or datetime.utcnow()
            elapsed = round((end - self.started_at).total_seconds(), 2)
        return {
            "task_id": self.task_id,
            "name": self.name,
            "task_type": self.task_type,
            "state": self.state.value,
            "progress": self.progress,
            "priority": self.priority.value,
            "created_by": self.created_by,
            "created_at": self.created_at.isoformat(),
            "started_at": self.started_at.isoformat() if self.started_at else
None,
            "completed_at": self.completed_at.isoformat() if self.completed_at
else None,
            "elapsed_seconds": elapsed,
            "error_message": self.error_message,
        }


class TaskScheduler:
    """任务调度器，管理分析任务的排队和执行"""

    def __init__(self, max_workers: int = 4, max_queue_size: int = 100):
        self.max_workers = max_workers
        self._task_queue = queue.PriorityQueue(maxsize=max_queue_size)
        self._tasks: Dict[str, AnalysisTask] = {}
        self._running_count = 0
        self._lock = threading.Lock()
        self._handlers: Dict[str, Callable] = {}

    def register_handler(self, task_type: str, handler: Callable):
        """注册任务类型对应的处理函数"""
        self._handlers[task_type] = handler
```

```
    def submit(self, task: AnalysisTask) -> str:
        """提交任务到队列"""
        if self._task_queue.full():
            raise RuntimeError("任务队列已满，请稍后再试")

        with self._lock:
            self._tasks[task.task_id] = task
            # 优先级越高数字越大，queue 取最小值，所以取负
            self._task_queue.put((-task.priority.value, task.created_at,
task.task_id))

        return task.task_id

    def get_task_status(self, task_id: str) -> Optional[Dict]:
        """查询任务状态"""
        task = self._tasks.get(task_id)
        if not task:
            return None
        return task.to_dict()

    def cancel_task(self, task_id: str) -> bool:
        """取消排队中的任务"""
        task = self._tasks.get(task_id)
        if not task:
            return False
        if task.state == TaskState.QUEUED:
            task.state = TaskState.CANCELLED
            task.completed_at = datetime.utcnow()
            return True
        return False

    def list_tasks(self, state: str = None, limit: int = 50) -> List[Dict]:
        """列出任务"""
        tasks = list(self._tasks.values())
        if state:
            tasks = [t for t in tasks if t.state.value == state]
        tasks.sort(key=lambda t: t.created_at, reverse=True)
        return [t.to_dict() for t in tasks[:limit]]

    def get_queue_stats(self) -> Dict:
        """获取队列统计"""
        with self._lock:
            states = {}
            for t in self._tasks.values():
                s = t.state.value
                states[s] = states.get(s, 0) + 1
            return {
                "queue_size": self._task_queue.qsize(),
                "running": self._running_count,
                "max_workers": self.max_workers,
                "total_tasks": len(self._tasks),
                "by_state": states,
            }

    async def execute_task(self, task_id: str):
```

```
        """执行单个任务"""
        task = self._tasks.get(task_id)
        if not task or task.state != TaskState.QUEUED:
            return

        handler = self._handlers.get(task.task_type)
        if not handler:
            task.state = TaskState.FAILED
            task.error_message = f"未注册的任务类型: {task.task_type}"
            task.completed_at = datetime.utcnow()
            return

        task.state = TaskState.RUNNING
        task.started_at = datetime.utcnow()
        with self._lock:
            self._running_count += 1

        try:
            result = await handler(task.params, task)
            task.result = result
            task.state = TaskState.COMPLETED
            task.progress = 100.0
        except TimeoutError:
            task.state = TaskState.TIMEOUT
            task.error_message = f"任务超时({task.timeout_seconds}秒)"
        except Exception as e:
            task.state = TaskState.FAILED
            task.error_message = str(e)
        finally:
            task.completed_at = datetime.utcnow()
            with self._lock:
                self._running_count -= 1

    def cleanup_old_tasks(self, days: int = 7):
        """清理已完成的历史任务"""
        cutoff = datetime.utcnow() - timedelta(days=days)
        to_remove = []
        for tid, task in self._tasks.items():
            if task.state in (TaskState.COMPLETED, TaskState.FAILED,
TaskState.CANCELLED):
                if task.completed_at and task.completed_at < cutoff:
                    to_remove.append(tid)
        for tid in to_remove:
            del self._tasks[tid]
        return len(to_remove)


task_scheduler = TaskScheduler(max_workers=4, max_queue_size=100)

# -*- coding: utf-8 -*-
"""
数据导出服务
"""
import csv
import io
import json
```

```
from datetime import datetime
from typing import List, Dict, Optional


class DataExportService:
    """数据导出工具，支持 CSV 和 JSON 格式"""

    def export_to_csv(self, data: List[Dict], columns: List[str] = None,
                      filename_prefix: str = "export") -> Dict:
        """将数据导出为 CSV 格式"""
        if not data:
            return {"error": "无数据可导出"}

        if columns is None:
            columns = list(data[0].keys())

        output = io.StringIO()
        writer = csv.DictWriter(output, fieldnames=columns,
extrasaction='ignore')
        writer.writeheader()

        for row in data:
            clean_row = {}
            for col in columns:
                val = row.get(col, "")
                if isinstance(val, datetime):
                    clean_row[col] = val.strftime("%Y-%m-%d %H:%M:%S")
                elif isinstance(val, (dict, list)):
                    clean_row[col] = json.dumps(val, ensure_ascii=False)
                else:
                    clean_row[col] = val
            writer.writerow(clean_row)

        content = output.getvalue()
        output.close()

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"{filename_prefix}_{timestamp}.csv"

        return {
            "filename": filename,
            "content": content,
            "content_type": "text/csv",
            "rows": len(data),
            "columns": columns
        }

    def export_to_json(self, data: List[Dict], filename_prefix: str = "export",
                       pretty: bool = True) -> Dict:
        """将数据导出为 JSON 格式"""
        if not data:
            return {"error": "无数据可导出"}

        indent = 2 if pretty else None
        content = json.dumps(data, ensure_ascii=False, indent=indent,
default=str)
```

```
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"{filename_prefix}_{timestamp}.json"

        return {
            "filename": filename,
            "content": content,
            "content_type": "application/json",
            "rows": len(data),
        }

    def export_timeseries_data(self, points: List, dataset_name: str) -> Dict:
        """导出时序数据点"""
        rows = []
        for p in points:
            rows.append({
                "timestamp": p.timestamp.strftime("%Y-%m-%d %H:%M:%S") if
hasattr(p, 'timestamp') else str(p.get("timestamp", "")),
                "value": p.value if hasattr(p, 'value') else p.get("value", 0),
                "quality": p.quality if hasattr(p, 'quality') else
p.get("quality", 100),
            })

        return self.export_to_csv(
            data=rows,
            columns=["timestamp", "value", "quality"],
            filename_prefix=f"timeseries_{dataset_name}"
        )

    def export_defect_results(self, results: List, batch_no: str = None) ->
Dict:
        """导出缺陷检测结果"""
        rows = []
        for r in results:
            rows.append({
                "sample_id": str(r.sample_id) if hasattr(r, 'sample_id') else
"",
                "has_defect": "是" if r.has_defect else "否",
                "confidence": round(r.confidence, 4) if r.confidence else 0,
                "defect_type": str(r.defect_type_id) if r.defect_type_id else
"无",
                "model_name": r.model_name or "",
                "created_at": r.created_at.strftime("%Y-%m-%d %H:%M:%S") if
r.created_at else "",
            })

        prefix = f"defect_{batch_no}" if batch_no else "defect_results"
        return self.export_to_csv(
            data=rows,
            columns=["sample_id", "has_defect", "confidence", "defect_type",
"model_name", "created_at"],
            filename_prefix=prefix
        )

    def export_anomaly_report(self, anomalies: List, dataset_name: str) ->
Dict:
```

```
        """导出异常检测报告"""
        rows = []
        for a in anomalies:
            rows.append({
                "timestamp": a.start_time.strftime("%Y-%m-%d %H:%M:%S") if
hasattr(a, 'start_time') else "",
                "anomaly_type": a.anomaly_type or "",
                "severity": a.severity or "medium",
                "score": round(a.score, 4) if a.score else 0,
                "description": a.description or "",
                "is_confirmed": "是" if a.is_confirmed else "否",
            })

        return self.export_to_csv(
            data=rows,
            columns=["timestamp", "anomaly_type", "severity", "score",
"description", "is_confirmed"],
            filename_prefix=f"anomalies_{dataset_name}"
        )


data_export_service = DataExportService()

# -*- coding: utf-8 -*-
"""
设备健康评估服务
"""
import math
from datetime import datetime, timedelta
from typing import Dict, List, Optional


class DeviceHealthService:
    """设备健康度计算和预测"""

    # 各因子权重
    WEIGHTS = {
        "age_factor": 0.15,
        "maintenance_factor": 0.25,
        "error_factor": 0.30,
        "utilization_factor": 0.15,
        "environment_factor": 0.15,
    }

    def calculate_health_score(self, device_info: Dict) -> Dict:
        """综合计算设备健康度评分"""

        age_score = self._calc_age_score(
            created_at=device_info.get("created_at"),
            expected_life_years=device_info.get("expected_life_years", 10)
        )

        maint_score = self._calc_maintenance_score(
            last_maintenance=device_info.get("last_maintenance_at"),

maintenance_interval_days=device_info.get("maintenance_interval_days", 90)
```

```
        )

        error_score = self._calc_error_score(
            error_count_30d=device_info.get("error_count_30d", 0),
            critical_errors_30d=device_info.get("critical_errors_30d", 0)
        )

        util_score = self._calc_utilization_score(
            utilization_rate=device_info.get("utilization_rate", 0.5)
        )

        env_score = self._calc_environment_score(
            temperature_deviation=device_info.get("temperature_deviation", 0),
            humidity_deviation=device_info.get("humidity_deviation", 0)
        )

        total = (
            self.WEIGHTS["age_factor"] * age_score +
            self.WEIGHTS["maintenance_factor"] * maint_score +
            self.WEIGHTS["error_factor"] * error_score +
            self.WEIGHTS["utilization_factor"] * util_score +
            self.WEIGHTS["environment_factor"] * env_score
        )

        total = max(0, min(100, total))

        if total >= 85:
            level = "excellent"
            recommendation = "设备状态良好，按计划维护即可"
        elif total >= 70:
            level = "good"
            recommendation = "设备状态正常，注意关注异常趋势"
        elif total >= 50:
            level = "fair"
            recommendation = "建议安排预防性维护，检查关键部件"
        elif total >= 30:
            level = "poor"
            recommendation = "设备健康度较低，建议尽快安排维修"
        else:
            level = "critical"
            recommendation = "设备状况危急，建议立即停机检修"

        return {
            "health_score": round(total, 1),
            "level": level,
            "recommendation": recommendation,
            "factors": {
                "age": round(age_score, 1),
                "maintenance": round(maint_score, 1),
                "error": round(error_score, 1),
                "utilization": round(util_score, 1),
                "environment": round(env_score, 1),
            }
        }

    def _calc_age_score(self, created_at, expected_life_years: int) -> float:
```

```
        if not created_at:
            return 80.0
        if isinstance(created_at, str):
            created_at = datetime.fromisoformat(created_at)
        age_days = (datetime.utcnow() - created_at).days
        age_years = age_days / 365.25
        ratio = age_years / expected_life_years
        if ratio <= 0.3:
            return 100.0
        elif ratio <= 0.7:
            return 100 - (ratio - 0.3) * 75
        else:
            return max(10, 100 - ratio * 80)

    def _calc_maintenance_score(self, last_maintenance,
maintenance_interval_days: int) -> float:
        if not last_maintenance:
            return 40.0
        if isinstance(last_maintenance, str):
            last_maintenance = datetime.fromisoformat(last_maintenance)
        days_since = (datetime.utcnow() - last_maintenance).days
        ratio = days_since / maintenance_interval_days
        if ratio <= 0.8:
            return 100.0
        elif ratio <= 1.0:
            return 100 - (ratio - 0.8) * 100
        elif ratio <= 1.5:
            return max(20, 80 - (ratio - 1.0) * 100)
        else:
            return max(5, 30 - (ratio - 1.5) * 50)

    def _calc_error_score(self, error_count_30d: int, critical_errors_30d: int)
-> float:
        if critical_errors_30d >= 5:
            return 10.0
        elif critical_errors_30d >= 2:
            return 30.0

        if error_count_30d == 0:
            return 100.0
        elif error_count_30d <= 3:
            return 90 - error_count_30d * 5
        elif error_count_30d <= 10:
            return 75 - (error_count_30d - 3) * 5
        else:
            return max(15, 40 - (error_count_30d - 10) * 3)

    def _calc_utilization_score(self, utilization_rate: float) -> float:
        if 0.4 <= utilization_rate <= 0.85:
            return 100.0
        elif utilization_rate < 0.4:
            return max(50, 100 - (0.4 - utilization_rate) * 200)
        else:
            return max(40, 100 - (utilization_rate - 0.85) * 300)

```

```
    def _calc_environment_score(self, temperature_deviation: float,
humidity_deviation: float) -> float:
        temp_penalty = min(50, abs(temperature_deviation) * 5)
        humid_penalty = min(30, abs(humidity_deviation) * 3)
        return max(10, 100 - temp_penalty - humid_penalty)

    def predict_remaining_life(self, health_history: List[Dict]) -> Dict:
        """根据健康度历史趋势预测剩余寿命"""
        if len(health_history) < 5:
            return {"predicted_days": None, "confidence": "low", "message": "
历史数据不足"}

        scores = [h["health_score"] for h in health_history]
        n = len(scores)
        x_mean = (n - 1) / 2.0
        y_mean = sum(scores) / n

        numerator = sum((i - x_mean) * (scores[i] - y_mean) for i in range(n))
        denominator = sum((i - x_mean) ** 2 for i in range(n))

        if denominator == 0 or numerator >= 0:
            return {"predicted_days": None, "confidence": "low", "message": "
健康度无明显下降趋势"}

        slope = numerator / denominator
        intercept = y_mean - slope * x_mean

        last_score = scores[-1]
        threshold = 30.0
        if last_score <= threshold:
            return {"predicted_days": 0, "confidence": "high", "message": "已
达维修阈值"}

        points_to_threshold = (threshold - intercept) / slope - (n - 1)
        interval_days = 1
        if len(health_history) >= 2:
            first_time = health_history[0].get("recorded_at")
            last_time = health_history[-1].get("recorded_at")
            if first_time and last_time:
                if isinstance(first_time, str):
                    first_time = datetime.fromisoformat(first_time)
                if isinstance(last_time, str):
                    last_time = datetime.fromisoformat(last_time)
                total_days = (last_time - first_time).days
                if total_days > 0:
                    interval_days = total_days / (n - 1)

        predicted_days = max(0, int(points_to_threshold * interval_days))

        if n >= 30:
            confidence = "high"
        elif n >= 15:
            confidence = "medium"
        else:
            confidence = "low"

```

```
        return {
            "predicted_days": predicted_days,
            "confidence": confidence,
            "current_score": round(last_score, 1),
            "decline_rate": round(abs(slope) / interval_days, 4),
            "threshold": threshold,
            "message": f"预计约{predicted_days}天后需要维修"
        }


device_health_service = DeviceHealthService()

# -*- coding: utf-8 -*-
"""
分析报告生成器
"""
import uuid
from datetime import datetime
from typing import Dict, List, Optional


class ReportGenerator:
    """自动化分析报告生成"""

    def generate_timeseries_report(self, dataset_info: Dict, analysis_result:
Dict,
                                   anomalies: List[Dict] = None) -> Dict:
        """生成时序分析报告"""
        report_id = str(uuid.uuid4())
        now = datetime.utcnow()

        stats = analysis_result.get("statistics", {})
        trend = analysis_result.get("trend", {})

        sections = []

        sections.append({
            "title": "1. 数据集概览",
            "content": (
                f"数据集名称：{dataset_info.get('name', '未知')}\n"
                f"数据源：{dataset_info.get('source_type', '未知')}\n"
                f"数据点数：{dataset_info.get('point_count', 0)}\n"
                f"时间范围：{dataset_info.get('start_time', '')} 至
{dataset_info.get('end_time', '')}\n"
                f"分析时间：{now.strftime('%Y-%m-%d %H:%M:%S')}"
            )
        })

        if stats:
            sections.append({
                "title": "2. 统计分析",
                "content": (
                    f"均值：{stats.get('mean', 0)}\n"
                    f"标准差：{stats.get('std', 0)}\n"
                    f"最小值：{stats.get('min', 0)}\n"
```

```
                    f"最大值：{stats.get('max', 0)}\n"
                    f"中位数：{stats.get('median', 0)}\n"
                    f"变异系数：{stats.get('cv', 0)}"
                )
            })

        if trend:
            direction_map = {"rising": "上升", "falling": "下降", "stable": "平稳"}
            sections.append({
                "title": "3. 趋势分析",
                "content": (
                    f"趋势方向：{direction_map.get(trend.get('direction', ''), '未知')}\n"
                    f"斜率：{trend.get('slope', 0)}\n"
                    f"拟合优度(R²)：{trend.get('r_squared', 0)}\n"
                    f"预测下一值：{trend.get('predicted_next', 0)}"
                )
            })

        if anomalies:
            anomaly_count = len(anomalies)
            high_severity = sum(1 for a in anomalies if a.get("severity") == "high")
            sections.append({
                "title": "4. 异常检测",
                "content": (
                    f"检出异常数：{anomaly_count}\n"
                    f"高严重度：{high_severity}\n"
                    f"异常率：{round(anomaly_count / max(dataset_info.get('point_count', 1), 1) * 100, 2)}%"
                )
            })

        conclusions = []
        if stats.get("cv", 0) > 0.3:
            conclusions.append("数据波动较大，建议检查传感器稳定性")
        if trend and trend.get("direction") == "rising" and trend.get("slope", 0) > 1:
            conclusions.append("数据呈明显上升趋势，需关注是否接近阈值")
        if anomalies and len(anomalies) > 5:
            conclusions.append("异常事件较多，建议排查设备运行状况")
        if not conclusions:
            conclusions.append("各项指标正常，建议持续监测")

        sections.append({
            "title": "5. 结论与建议",
            "content": "\n".join(f"- {c}" for c in conclusions)
        })

        return {
            "report_id": report_id,
            "title": f"时序分析报告 - {dataset_info.get('name', '')}",
            "report_type": "timeseries_analysis",
```

```
            "created_at": now.isoformat(),
            "sections": sections,
            "summary": "；".join(conclusions)
        }

    def generate_defect_report(self, batch_info: Dict, detection_results:
List[Dict]) -> Dict:
        """生成缺陷检测报告"""
        report_id = str(uuid.uuid4())
        now = datetime.utcnow()

        total = len(detection_results)
        defect_count = sum(1 for r in detection_results if r.get("has_defect"))
        normal_count = total - defect_count
        pass_rate = round(normal_count / total * 100, 2) if total > 0 else
100.0

        type_counts = {}
        for r in detection_results:
            if r.get("has_defect"):
                dt = r.get("defect_type_name", "未分类")
                type_counts[dt] = type_counts.get(dt, 0) + 1

        confidence_vals = [r.get("confidence", 0) for r in detection_results
if r.get("confidence")]
        avg_confidence = sum(confidence_vals) / len(confidence_vals) if
confidence_vals else 0

        sections = [
            {
                "title": "1. 批次信息",
                "content": (
                    f"批次号：{batch_info.get('batch_no', '')}\n"
                    f"产品类型：{batch_info.get('product_type', '')}\n"
                    f"检测时间：{now.strftime('%Y-%m-%d %H:%M:%S')}"
                )
            },
            {
                "title": "2. 检测结果汇总",
                "content": (
                    f"检测总数：{total}\n"
                    f"合格数：{normal_count}\n"
                    f"缺陷数：{defect_count}\n"
                    f"合格率：{pass_rate}%\n"
                    f"平均置信度：{round(avg_confidence, 4)}"
                )
            },
        ]

        if type_counts:
            type_lines = "\n".join(f"  {name}: {count}件" for name, count in
                                   sorted(type_counts.items(), key=lambda x:
x[1], reverse=True))
            sections.append({
                "title": "3. 缺陷类型分布",
```

```
                "content": type_lines
            })

        if pass_rate >= 99:
            conclusion = "批次质量优秀，合格率达 99%以上"
        elif pass_rate >= 95:
            conclusion = "批次质量良好，建议关注缺陷类型分布"
        elif pass_rate >= 90:
            conclusion = "批次质量一般，建议排查主要缺陷原因"
        else:
            conclusion = "批次质量不达标，建议停线排查问题并加强检测"

        sections.append({
            "title": "4. 结论",
            "content": conclusion
        })

        return {
            "report_id": report_id,
            "title": f"缺陷检测报告 - 批次{batch_info.get('batch_no', '')}",
            "report_type": "defect_detection",
            "created_at": now.isoformat(),
            "sections": sections,
            "pass_rate": pass_rate,
            "summary": conclusion
        }


report_generator = ReportGenerator()

# -*- coding: utf-8 -*-
"""
简易内存缓存服务
"""
import time
import threading
from typing import Any, Optional, Dict


class CacheEntry:
    """缓存条目"""
    __slots__ = ('value', 'expire_at', 'created_at', 'access_count')

    def __init__(self, value: Any, ttl: int):
        self.value = value
        self.created_at = time.time()
        self.expire_at = self.created_at + ttl if ttl > 0 else 0
        self.access_count = 0

    def is_expired(self) -> bool:
        if self.expire_at == 0:
            return False
        return time.time() > self.expire_at


class MemoryCache:
```

```
    """线程安全的内存缓存"""

    def __init__(self, max_size: int = 1000, default_ttl: int = 300):
        self._store: Dict[str, CacheEntry] = {}
        self._lock = threading.RLock()
        self._max_size = max_size
        self._default_ttl = default_ttl
        self._hit_count = 0
        self._miss_count = 0

    def get(self, key: str, default: Any = None) -> Any:
        with self._lock:
            entry = self._store.get(key)
            if entry is None:
                self._miss_count += 1
                return default
            if entry.is_expired():
                del self._store[key]
                self._miss_count += 1
                return default
            entry.access_count += 1
            self._hit_count += 1
            return entry.value

    def set(self, key: str, value: Any, ttl: int = None):
        if ttl is None:
            ttl = self._default_ttl
        with self._lock:
            if len(self._store) >= self._max_size and key not in self._store:
                self._evict()
            self._store[key] = CacheEntry(value, ttl)

    def delete(self, key: str) -> bool:
        with self._lock:
            if key in self._store:
                del self._store[key]
                return True
            return False

    def clear(self):
        with self._lock:
            self._store.clear()
            self._hit_count = 0
            self._miss_count = 0

    def exists(self, key: str) -> bool:
        with self._lock:
            entry = self._store.get(key)
            if entry is None:
                return False
            if entry.is_expired():
                del self._store[key]
                return False
            return True

    def get_or_set(self, key: str, factory, ttl: int = None) -> Any:
```

```
        """获取缓存值，不存在时通过 factory 创建"""
        value = self.get(key)
        if value is not None:
            return value
        value = factory()
        self.set(key, value, ttl)
        return value

    def _evict(self):
        """淘汰策略：先清过期，再按 LFU 淘汰"""
        expired_keys = [k for k, v in self._store.items() if v.is_expired()]
        for k in expired_keys:
            del self._store[k]

        if len(self._store) >= self._max_size:
            sorted_entries = sorted(self._store.items(), key=lambda x:
x[1].access_count)
            remove_count = len(self._store) // 5
            for k, _ in sorted_entries[:remove_count]:
                del self._store[k]

    def get_stats(self) -> Dict:
        with self._lock:
            total = self._hit_count + self._miss_count
            return {
                "size": len(self._store),
                "max_size": self._max_size,
                "hit_count": self._hit_count,
                "miss_count": self._miss_count,
                "hit_rate": round(self._hit_count / total * 100, 2) if total >
0 else 0,
                "default_ttl": self._default_ttl,
            }

    def cleanup_expired(self) -> int:
        with self._lock:
            expired = [k for k, v in self._store.items() if v.is_expired()]
            for k in expired:
                del self._store[k]
            return len(expired)


cache = MemoryCache(max_size=2000, default_ttl=300)

# 预定义缓存 key 生成器
def cache_key(*parts) -> str:
    return ":".join(str(p) for p in parts)

# -*- coding: utf-8 -*-
"""
批量数据处理器
"""
import csv
import io
import os
import uuid
```

```
from datetime import datetime
from typing import Dict, List, Optional, Tuple


class CSVDataParser:
    """CSV 时序数据解析器"""

    SUPPORTED_COLUMNS = {
        "timestamp": ["timestamp", "time", "datetime", "date", "时间", "时间戳
"],
        "value": ["value", "val", "data", "reading", "数值", "值"],
        "quality": ["quality", "qual", "flag", "质量", "标志"],
    }

    def parse_csv_content(self, content: str, delimiter: str = ",") ->
Tuple[List[Dict], Dict]:
        """解析 CSV 字符串内容"""
        reader = csv.DictReader(io.StringIO(content), delimiter=delimiter)
        headers = reader.fieldnames or []

        col_map = self._detect_columns(headers)
        if "timestamp" not in col_map:
            return [], {"error": "未找到时间戳列", "headers": headers}
        if "value" not in col_map:
            return [], {"error": "未找到数值列", "headers": headers}

        rows = []
        errors = []
        for line_num, row in enumerate(reader, start=2):
            try:
                ts_str = row.get(col_map["timestamp"], "").strip()
                val_str = row.get(col_map["value"], "").strip()

                if not ts_str or not val_str:
                    continue

                ts = self._parse_timestamp(ts_str)
                val = float(val_str)

                quality = 100
                if "quality" in col_map:
                    q_str = row.get(col_map["quality"], "100").strip()
                    try:
                        quality = int(float(q_str))
                    except (ValueError, TypeError):
                        quality = 100

                rows.append({
                    "timestamp": ts,
            cycle_means.append(sum(segment) / len(segment))

        overall_mean = sum(cycle_means) / len(cycle_means)
        cycle_variance = sum((m - overall_mean) ** 2 for m in cycle_means) /
len(cycle_means)

        seasonal_indices = []
```

```
        for pos in range(period):
            pos_values = [values[c * period + pos] for c in range(cycles) if c
* period + pos < len(values)]
            avg = sum(pos_values) / len(pos_values) if pos_values else 0
            seasonal_indices.append(round(avg / overall_mean, 4) if
overall_mean != 0 else 1.0)

        return {
            "has_seasonality": cycle_variance > 0.01,
            "period": period,
            "seasonal_indices": seasonal_indices,
            "cycle_count": cycles
        }


timeseries_service = TimeseriesAnalysisService()

# -*- coding: utf-8 -*-
"""
缺陷统计聚合服务
"""

class DefectStatisticsService:
    """缺陷统计分析"""

    async def get_summary(self, db, device_id: Optional[str] = None,
                          start_date: Optional[datetime] = None,
                          end_date: Optional[datetime] = None) -> Dict:
        """获取缺陷检测汇总"""
        from app.models.defect import DefectSample, DefectResult, DefectType

        query = select(DefectResult)
        conditions = []

        if start_date:
            conditions.append(DefectResult.created_at >= start_date)
        if end_date:
            conditions.append(DefectResult.created_at <= end_date)
        if device_id:
            sample_ids = await self._get_device_sample_ids(db, device_id)
            if sample_ids:
                conditions.append(DefectResult.sample_id.in_(sample_ids))

        if conditions:
            combined = conditions[0]
            for c in conditions[1:]:
                combined = combined & c
            query = query.where(combined)

        result = await db.execute(query)
        results = result.scalars().all()

        total = len(results)
        defect_count = sum(1 for r in results if r.has_defect)
        normal_count = total - defect_count
```

```
        avg_confidence = sum(r.confidence for r in results if r.confidence) /
total if total > 0 else 0

        type_dist = {}
        for r in results:
            if r.has_defect and r.defect_type_id:
                tid = str(r.defect_type_id)
                type_dist[tid] = type_dist.get(tid, 0) + 1

        return {
            "total_inspected": total,
            "defect_count": defect_count,
            "normal_count": normal_count,
            "defect_rate": round(defect_count / total * 100, 2) if total > 0
else 0,
            "avg_confidence": round(avg_confidence, 4),
            "type_distribution": type_dist
        }

    async def get_severity_distribution(self, db) -> List[Dict]:
        """按严重等级统计缺陷分布"""
        from app.models.defect import DefectType, DefectResult

        types_result = await
db.execute(select(DefectType).where(DefectType.is_active == True))
        defect_types = types_result.scalars().all()

        distribution = []
        for dt in defect_types:
            count_result = await db.execute(
                select(func.count(DefectResult.id)).where(
                    DefectResult.defect_type_id == dt.id,
                    DefectResult.has_defect == True
                )
            )
            count = count_result.scalar() or 0
            distribution.append({
                "type_id": str(dt.id),
                "type_name": dt.name,
                "type_code": dt.code,
                "category": dt.category,
                "severity": dt.severity,
                "count": count
            })

        distribution.sort(key=lambda x: x["count"], reverse=True)
        return distribution

    async def get_batch_quality_report(self, db, batch_id: str) -> Dict:
        """按批次生成质量检测报告"""
        from app.models.defect import DefectSample, DefectResult
        from app.models.industrial import Batch

        batch_result = await db.execute(select(Batch).where(Batch.id ==
batch_id))
        batch = batch_result.scalar_one_or_none()
```

```
        if not batch:
            return {"error": "批次不存在"}

        samples_result = await db.execute(
            select(DefectSample).where(DefectSample.batch_id == batch_id)
        )
        samples = samples_result.scalars().all()
        sample_ids = [s.id for s in samples]

        if not sample_ids:
            return {
                "batch_no": batch.batch_no,
                "total_samples": 0,
                "inspected": 0,
                "defect_count": 0,
                "pass_rate": 100.0
            }

        results_query = await db.execute(
            select(DefectResult).where(DefectResult.sample_id.in_(sample_ids))
        )
        all_results = results_query.scalars().all()

        inspected = len(all_results)
        defect_count = sum(1 for r in all_results if r.has_defect)
        pass_rate = round((inspected - defect_count) / inspected * 100, 2) if
inspected > 0 else 100.0

        defect_details = []
        for r in all_results:
            if r.has_defect:
                defect_details.append({
                    "sample_id": str(r.sample_id),
                    "confidence": r.confidence,
                    "bbox": r.bbox,
                    "model_name": r.model_name,
                    "created_at": r.created_at.isoformat() if r.created_at
else None
                })

        return {
            "batch_no": batch.batch_no,
            "product_type": batch.product_type,
            "total_samples": len(samples),
            "inspected": inspected,
            "defect_count": defect_count,
            "pass_rate": pass_rate,
            "defect_details": defect_details
        }

    async def _get_device_sample_ids(self, db, device_id: str) -> List:
        from app.models.defect import DefectSample
        result = await db.execute(
            select(DefectSample.id).where(DefectSample.device_id == device_id)
        )
        return [row[0] for row in result.all()]
```

```
defect_stats_service = DefectStatisticsService()

# -*- coding: utf-8 -*-
"""
审计日志服务
"""

class AuditService:
    """审计日志记录和查询"""

    SENSITIVE_FIELDS = {"password", "token", "secret", "hashed_password",
"access_token", "refresh_token"}

    @classmethod
    async def log(
        cls,
        db,
        user_id,
        username: str,
        action: str,
        resource_type: str = None,
        resource_id: str = None,
        description: str = "",
        old_value: Dict = None,
        new_value: Dict = None,
        request=None
    ):
        """记录审计日志"""
        from app.models.system import AuditLog
        import uuid

        ip_address = None
        user_agent = None
        if request:
            ip_address = cls._extract_ip(request)
            user_agent = request.headers.get("user-agent", "")

        log_entry = AuditLog(
            id=uuid.uuid4(),
            user_id=user_id,
            username=username,
            action=action,
            resource_type=resource_type,
            resource_id=resource_id,
            details={"description": description},
            old_value=cls._filter_sensitive(old_value) if old_value else None,
            new_value=cls._filter_sensitive(new_value) if new_value else None,
            ip_address=ip_address,
            user_agent=user_agent[:500] if user_agent else None,
            status="success"
        )

        db.add(log_entry)
        try:
```

```
            await db.flush()
        except Exception:
            pass

    @classmethod
    def _extract_ip(cls, request) -> str:
        """从请求中提取客户端 IP"""
        forwarded = request.headers.get("x-forwarded-for")
        if forwarded:
            return forwarded.split(",")[0].strip()
        real_ip = request.headers.get("x-real-ip")
        if real_ip:
            return real_ip
        if hasattr(request, "client") and request.client:
            return request.client.host
        return "unknown"

    @classmethod
    def _filter_sensitive(cls, data: Dict) -> Dict:
        """过滤敏感字段"""
        if not isinstance(data, dict):
            return data
        filtered = {}
        for key, value in data.items():
            if key.lower() in cls.SENSITIVE_FIELDS:
                filtered[key] = "***"
            elif isinstance(value, dict):
                filtered[key] = cls._filter_sensitive(value)
            else:
                filtered[key] = value
        return filtered

    @classmethod
    async def query_logs(
        cls, db, page: int = 1, page_size: int = 20,
        user_id=None, action: str = None,
        resource_type: str = None, start_date=None, end_date=None
    ) -> Dict:
        """查询审计日志"""
        from app.models.system import AuditLog

        query = select(AuditLog)
        count_query = select(func.count(AuditLog.id))

        conditions = []
        if user_id:
            conditions.append(AuditLog.user_id == user_id)
        if action:
            conditions.append(AuditLog.action == action)
        if resource_type:
            conditions.append(AuditLog.resource_type == resource_type)
        if start_date:
            conditions.append(AuditLog.created_at >= start_date)
        if end_date:
            conditions.append(AuditLog.created_at <= end_date)

```

```
        if conditions:
            combined = conditions[0]
            for c in conditions[1:]:
                combined = combined & c
            query = query.where(combined)
            count_query = count_query.where(combined)

        total_result = await db.execute(count_query)
        total = total_result.scalar() or 0

        query = query.order_by(AuditLog.created_at.desc())
        query = query.offset((page - 1) * page_size).limit(page_size)

        result = await db.execute(query)
        logs = result.scalars().all()

        total_pages = (total + page_size - 1) // page_size

        return {
            "items": [
                {
                    "id": str(log.id),
                    "username": log.username,
                    "action": log.action,
                    "resource_type": log.resource_type,
                    "resource_id": log.resource_id,
                    "details": log.details,
                    "ip_address": log.ip_address,
                    "status": log.status,
                    "created_at": log.created_at.isoformat() if log.created_at
else None
                }
                for log in logs
            ],
            "total": total,
            "page": page,
            "page_size": page_size,
            "total_pages": total_pages
        }

# -*- coding: utf-8 -*-
"""
通知服务
"""

class NotificationService:
    """系统通知管理"""

    @classmethod
    async def send(cls, db, user_id, title: str, content: str,
                   notification_type: str = "info",
                   related_type: str = None, related_id=None,
                   extra_data: Dict = None):
        """发送通知给用户"""
        from app.models.system import Notification
        import uuid
```

```python

        notification = Notification(
            id=uuid.uuid4(),
            user_id=user_id,
            title=title,
            content=content,
            notification_type=notification_type,
            related_type=related_type,
            related_id=related_id,
            extra_data=extra_data
        )
        db.add(notification)
        await db.flush()
        return notification

    @classmethod
    async def send_batch(cls, db, user_ids: List, title: str, content: str,
                         notification_type: str = "info"):
        """批量发送通知"""
        notifications = []
        for uid in user_ids:
            n = await cls.send(db, uid, title, content, notification_type)
            notifications.append(n)
        return notifications

    @classmethod
    async def mark_read(cls, db, notification_id: str, user_id):
        """标记通知为已读"""
        from app.models.system import Notification

        result = await db.execute(
            select(Notification).where(
                Notification.id == notification_id,
                Notification.user_id == user_id
            )
        )
        notification = result.scalar_one_or_none()
        if notification and not notification.is_read:
            notification.is_read = True
            notification.read_at = datetime.utcnow()
            await db.flush()
        return notification

    @classmethod
    async def mark_all_read(cls, db, user_id):
        """标记用户所有通知为已读"""
        from app.models.system import Notification

        result = await db.execute(
            select(Notification).where(
                Notification.user_id == user_id,
                Notification.is_read == False
            )
        )
        notifications = result.scalars().all()
        now = datetime.utcnow()
```

```
        for n in notifications:
            n.is_read = True
            n.read_at = now
        await db.flush()
        return len(notifications)

    @classmethod
    async def get_unread_count(cls, db, user_id) -> int:
        """获取未读通知数量"""
        from app.models.system import Notification

        result = await db.execute(
            select(func.count(Notification.id)).where(
                Notification.user_id == user_id,
                Notification.is_read == False
            )
        )
        return result.scalar() or 0

    @classmethod
    async def get_notifications(cls, db, user_id, page: int = 1,
                                page_size: int = 20,
                                unread_only: bool = False) -> Dict:
        """获取用户通知列表"""
        from app.models.system import Notification

        query = select(Notification).where(Notification.user_id == user_id)
        count_query =
select(func.count(Notification.id)).where(Notification.user_id == user_id)

        if unread_only:
            query = query.where(Notification.is_read == False)
            count_query = count_query.where(Notification.is_read == False)

        total_result = await db.execute(count_query)
        total = total_result.scalar() or 0

        query = query.order_by(Notification.created_at.desc())
        query = query.offset((page - 1) * page_size).limit(page_size)

        result = await db.execute(query)
        notifications = result.scalars().all()

        return {
            "items": [
                {
                    "id": str(n.id),
                    "title": n.title,
                    "content": n.content,
                    "type": n.notification_type,
                    "is_read": n.is_read,
                    "read_at": n.read_at.isoformat() if n.read_at else None,
                    "related_type": n.related_type,
                    "related_id": str(n.related_id) if n.related_id else None,
                    "created_at": n.created_at.isoformat() if n.created_at
else None
```

```
                }
                for n in notifications
            ],
            "total": total,
            "page": page,
            "page_size": page_size,
            "unread_count": await cls.get_unread_count(db, user_id)
        }


notification_service = NotificationService()

# -*- coding: utf-8 -*-
"""
文件存储服务
"""
import os
import hashlib
import uuid as uuid_mod
from pathlib import Path

class FileStorageService:
    """本地文件存储服务"""

    def __init__(self, base_dir: str = "./uploads"):
        self.base_dir = base_dir
        self._ensure_dirs()

    def _ensure_dirs(self):
        """确保上传目录存在"""
        dirs = ["defects", "timeseries", "reports", "avatars", "temp"]
        for d in dirs:
            full_path = os.path.join(self.base_dir, d)
            os.makedirs(full_path, exist_ok=True)

    def save_file(self, file_content: bytes, original_name: str,
                  category: str = "temp") -> Dict:
        """保存文件，返回文件信息"""
        ext = os.path.splitext(original_name)[1].lower() if "." in
original_name else ""
        file_id = str(uuid_mod.uuid4())
        safe_name = f"{file_id}{ext}"

        target_dir = os.path.join(self.base_dir, category)
        os.makedirs(target_dir, exist_ok=True)
        file_path = os.path.join(target_dir, safe_name)

        with open(file_path, "wb") as f:
            f.write(file_content)

        file_hash = hashlib.md5(file_content).hexdigest()
        file_size = len(file_content)

        return {
            "file_id": file_id,
            "file_name": safe_name,
```

```
            "original_name": original_name,
            "file_path": file_path,
            "file_size": file_size,
            "file_hash": file_hash,
            "extension": ext,
            "category": category
        }

    def delete_file(self, file_path: str) -> bool:
        """删除文件"""
        if os.path.exists(file_path):
            os.remove(file_path)
            return True
        return False

    def get_file_info(self, file_path: str) -> Optional[Dict]:
        """获取文件信息"""
        if not os.path.exists(file_path):
            return None
        stat = os.stat(file_path)
        return {
            "path": file_path,
            "size": stat.st_size,
            "created": datetime.fromtimestamp(stat.st_ctime).isoformat(),
            "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
        }

    def validate_image(self, file_content: bytes, max_size: int = 50 * 1024 *
1024) -> Tuple[bool, str]:
        """校验图片文件"""
        if len(file_content) > max_size:
            return False, f"文件大小超过限制({max_size // 1024 // 1024}MB)"

        if len(file_content) < 8:
            return False, "文件内容为空或过小"

        header = file_content[:8]
        if header[:2] == b'\xff\xd8':
            return True, "jpeg"
        elif header[:8] == b'\x89PNG\r\n\x1a\n':
            return True, "png"
        elif header[:4] == b'BM':
            return True, "bmp"
        elif header[:4] in (b'II*\x00', b'MM\x00*'):
            return True, "tiff"
        else:
            return False, "不支持的图片格式，仅支持 JPEG/PNG/BMP/TIFF"

    def get_storage_stats(self) -> Dict:
        """获取存储空间使用统计"""
        stats = {}
        total_size = 0
        total_files = 0

        for category in ["defects", "timeseries", "reports", "avatars",
"temp"]:
```

```
            cat_dir = os.path.join(self.base_dir, category)
            if not os.path.exists(cat_dir):
                stats[category] = {"files": 0, "size_mb": 0}
                continue
            cat_files = 0
            cat_size = 0
            for f in os.listdir(cat_dir):
                fp = os.path.join(cat_dir, f)
                if os.path.isfile(fp):
                    cat_files += 1
                    cat_size += os.path.getsize(fp)
            stats[category] = {
                "files": cat_files,
                "size_mb": round(cat_size / 1024 / 1024, 2)
            }
            total_files += cat_files
            total_size += cat_size

        stats["total"] = {
            "files": total_files,
            "size_mb": round(total_size / 1024 / 1024, 2)
        }
        return stats


file_storage = FileStorageService()

# -*- coding: utf-8 -*-
"""
仪表盘数据聚合服务
"""

class DashboardService:
    """仪表盘统计数据聚合"""

    async def get_overview(self, db) -> Dict:
        """获取系统总览数据"""
        from app.models.industrial import Device, ProductionLine
        from app.models.defect import DefectSample, DefectResult
        from app.models.timeseries import TimeseriesDataset, TimeseriesAnomaly

        device_count = await self._count(db, Device)
        online_count = await self._count_where(db, Device, Device.status ==
"online")
        offline_count = await self._count_where(db, Device, Device.status ==
"offline")
        warning_count = await self._count_where(db, Device, Device.status ==
"warning")

        line_count = await self._count(db, ProductionLine)
        sample_count = await self._count(db, DefectSample)
        defect_count = await self._count_where(db, DefectResult,
DefectResult.has_defect == True)
        dataset_count = await self._count(db, TimeseriesDataset)
        anomaly_count = await self._count(db, TimeseriesAnomaly)

```

```
        return {
            "devices": {
                "total": device_count,
                "online": online_count,
                "offline": offline_count,
                "warning": warning_count,
                "online_rate": round(online_count / device_count * 100, 1) if
device_count > 0 else 0
            },
            "production_lines": line_count,
            "defect_detection": {
                "total_samples": sample_count,
                "defect_found": defect_count,
                "defect_rate": round(defect_count / sample_count * 100, 2) if
sample_count > 0 else 0
            },
            "timeseries": {
                "datasets": dataset_count,
                "anomalies_detected": anomaly_count
            }
        }

    async def get_recent_alerts(self, db, limit: int = 10) -> List[Dict]:
        """获取最近的告警事件"""
        from app.models.system import Notification

        result = await db.execute(
            select(Notification)
            .where(Notification.notification_type == "alert")
            .order_by(Notification.created_at.desc())
            .limit(limit)
        )
        notifications = result.scalars().all()

        return [
            {
                "id": str(n.id),
                "title": n.title,
                "content": n.content,
                "type": n.notification_type,
                "is_read": n.is_read,
                "created_at": n.created_at.isoformat() if n.created_at else
None
            }
            for n in notifications
        ]

    async def get_defect_trend(self, db, days: int = 30) -> List[Dict]:
        """获取缺陷检测趋势"""
        from app.models.defect import DefectResult

        end_date = datetime.utcnow()
        start_date = end_date - timedelta(days=days)

        result = await db.execute(
            select(DefectResult)
```

```
            .where(DefectResult.created_at >= start_date)
            .order_by(DefectResult.created_at)
        )
        all_results = result.scalars().all()

        daily_stats = {}
        for r in all_results:
            day_key = r.created_at.strftime("%Y-%m-%d")
            if day_key not in daily_stats:
                daily_stats[day_key] = {"total": 0, "defect": 0}
            daily_stats[day_key]["total"] += 1
            if r.has_defect:
                daily_stats[day_key]["defect"] += 1

        trend = []
        current = start_date
        while current <= end_date:
            day_str = current.strftime("%Y-%m-%d")
            stats = daily_stats.get(day_str, {"total": 0, "defect": 0})
            rate = round(stats["defect"] / stats["total"] * 100, 2) if
stats["total"] > 0 else 0
            trend.append({
                "date": day_str,
                "total": stats["total"],
                "defect": stats["defect"],
                "rate": rate
            })
            current += timedelta(days=1)

        return trend

    async def get_device_health_ranking(self, db, limit: int = 20) ->
List[Dict]:
        """设备健康度排行"""
        from app.models.industrial import Device

        result = await db.execute(
            select(Device)
            .order_by(Device.health_score.asc())
            .limit(limit)
        )
        devices = result.scalars().all()

        ranking = []
        for idx, d in enumerate(devices, 1):
            score = d.health_score if d.health_score is not None else 100.0
            if score >= 85:
                level = "good"
            elif score >= 60:
                level = "fair"
            elif score >= 40:
                level = "poor"
            else:
                level = "critical"
            ranking.append({
                "rank": idx,
```

```
                "device_id": str(d.id),
                "device_name": d.name,
                "device_code": d.code,
                "health_score": score,
                "level": level,
                "status": d.status,
                "last_maintenance": d.last_maintenance_at.isoformat() if
d.last_maintenance_at else None
            })

        return ranking

    async def _count(self, db, model):
        result = await db.execute(select(func.count(model.id)))
        return result.scalar() or 0

    async def _count_where(self, db, model, condition):
        result = await
db.execute(select(func.count(model.id)).where(condition))
        return result.scalar() or 0


dashboard_service = DashboardService()

async def get_current_active_user(current_user = Depends(get_current_user)):
    """获取当前活跃用户"""
    if not current_user.is_active:
        raise UserDisabledError(current_user.username)
    return current_user

class PermissionChecker:
    """权限检查器"""

    def __init__(self, required_permissions: list[str] = None, required_roles:
list[str] = None):
        self.required_permissions = required_permissions or []
        self.required_roles = required_roles or []

    async def __call__(
        self,
        current_user = Depends(get_current_user),
        db: AsyncSession = Depends(get_db)
    ):
        from app.models.user import User
        from app.models.rbac import Role, Permission, RolePermission

        # 超级用户拥有所有权限
        if current_user.role and current_user.role.name == "superuser":
            return current_user

        # 检查角色
        if self.required_roles:
            if not current_user.role or current_user.role.name not in
self.required_roles:
                raise PermissionDeniedError(f"需要以下角色之一: {',
'.join(self.required_roles)}")
```

```

        # 检查权限
        if self.required_permissions:
            user_permissions = set()
            if current_user.role:
                for rp in current_user.role.role_permissions:
                    user_permissions.add(rp.permission.code)

            missing_permissions = set(self.required_permissions) -
user_permissions
            if missing_permissions:
                raise PermissionDeniedError(f"缺少权限: {',
'.join(missing_permissions)}")

        return current_user

# 常用权限检查器
require_admin = PermissionChecker(required_roles=["admin", "superuser"])
require_superuser = PermissionChecker(required_roles=["superuser"])

"""
Audit Middleware
审计中间件
"""
import time
import json
from typing import Callable, Optional
# 以下引入平台核心依赖模块（数据库、路由、校验、ORM 等）

from app.core.database import async_session_maker

class AuditMiddleware(BaseHTTPMiddleware):
    """审计日志中间件"""

    # 需要记录的路径前缀
    AUDIT_PATHS = [
        "/api/v1/auth/login",
        "/api/v1/auth/register",
        "/api/v1/auth/logout",
        "/api/v1/users",
        "/api/v1/defects",
        "/api/v1/timeseries",
        "/api/v1/industrial",
        "/api/v1/tasks",
    ]

    # 忽略的路径
    IGNORE_PATHS = [
        "/health",
        "/docs",
        "/openapi.json",
        "/redoc",
        "/static",
    ]

```

```
    # 敏感字段（不记录到日志）
    SENSITIVE_FIELDS = [
        "password",
        "token",
        "secret",
        "access_token",
        "refresh_token",
    ]

    async def dispatch(self, request: Request, call_next: Callable) ->
Response:
        """处理请求"""
        # 检查是否需要审计
        path = request.url.path

        if self._should_ignore(path):
            return await call_next(request)

        # 记录开始时间
        start_time = time.time()

        # 获取请求信息
        request_info = await self._extract_request_info(request)

        # 执行请求
        response = await call_next(request)

        # 计算处理时间
        process_time = time.time() - start_time

        # 如果需要审计，记录日志
        if self._should_audit(path, request.method):
            await self._log_request(
                request_info=request_info,
                status_code=response.status_code,
                process_time=process_time
            )

        # 添加处理时间头
        response.headers["X-Process-Time"] = str(round(process_time * 1000, 2))

        return response

    def _should_ignore(self, path: str) -> bool:
        """检查是否应该忽略该路径"""
        for ignore_path in self.IGNORE_PATHS:
            if path.startswith(ignore_path):
                return True
        return False

    def _should_audit(self, path: str, method: str) -> bool:
        """检查是否需要审计"""
        # 只审计非 GET 请求（修改操作）
        if method == "GET":
            return False
```

```
        for audit_path in self.AUDIT_PATHS:
            if path.startswith(audit_path):
                return True
        return False

    async def _extract_request_info(self, request: Request) -> dict:
        """提取请求信息"""
        info = {
            "method": request.method,
            "path": request.url.path,
            "query": str(request.query_params),
            "ip_address": self._get_client_ip(request),
            "user_agent": request.headers.get("user-agent", ""),
        }

        # 尝试获取用户 ID（从 JWT token）
        auth_header = request.headers.get("authorization", "")
        if auth_header.startswith("Bearer "):
            info["has_auth"] = True

        # 尝试读取请求体（仅针对 JSON 请求）
        content_type = request.headers.get("content-type", "")
        if "application/json" in content_type:
            try:
                body = await request.body()
                if body:
                    body_json = json.loads(body)
                    # 过滤敏感字段
                    filtered_body = self._filter_sensitive_data(body_json)
                    info["body"] = filtered_body
            except Exception:
                pass

        return info

    def _get_client_ip(self, request: Request) -> str:
        """获取客户端 IP"""
        # 检查代理头
        forwarded = request.headers.get("x-forwarded-for")
        if forwarded:
            return forwarded.split(",")[0].strip()

        real_ip = request.headers.get("x-real-ip")
        if real_ip:
            return real_ip

        # 直接连接的客户端
        if request.client:
            return request.client.host

        return "unknown"

    def _filter_sensitive_data(self, data: dict) -> dict:
        """过滤敏感数据"""
        if not isinstance(data, dict):
```

```
            return data

        filtered = {}
        for key, value in data.items():
            if any(sf in key.lower() for sf in self.SENSITIVE_FIELDS):
                filtered[key] = "***FILTERED***"
            elif isinstance(value, dict):
                filtered[key] = self._filter_sensitive_data(value)
            else:
                filtered[key] = value

        return filtered

    async def _log_request(
        self,
        request_info: dict,
        status_code: int,
        process_time: float
    ):
        """记录请求日志"""
        # 这里可以写入数据库或日志文件
        # 为简化实现，仅在控制台输出
        log_entry = {
            **request_info,
            "status_code": status_code,
            "process_time_ms": round(process_time * 1000, 2),
        }

        # 在实际项目中，这里会调用审计服务写入数据库
        # 由于中间件中获取数据库会话比较复杂，这里简化处理
        import logging
        logger = logging.getLogger("audit")
        logger.info(json.dumps(log_entry, ensure_ascii=False))

class RequestIdMiddleware(BaseHTTPMiddleware):
    """请求 ID 中间件"""

    async def dispatch(self, request: Request, call_next: Callable) ->
Response:
        import uuid

        # 生成或获取请求 ID
        request_id = request.headers.get("x-request-id")
        if not request_id:
            request_id = str(uuid.uuid4())

        # 存储到请求状态
        request.state.request_id = request_id

        # 执行请求
        response = await call_next(request)

        # 添加请求 ID 到响应头
        response.headers["X-Request-ID"] = request_id

```

```
        return response

class RateLimitMiddleware(BaseHTTPMiddleware):
    """简单的速率限制中间件"""

    def __init__(self, app, max_requests: int = 100, window_seconds: int = 60):
        super().__init__(app)
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self.request_counts = {}  # {ip: [(timestamp, count), ...]}

    async def dispatch(self, request: Request, call_next: Callable) ->
Response:
        # 引入平台 HTTP 响应模块

        client_ip = self._get_client_ip(request)
        current_time = time.time()

        # 清理过期记录
        self._cleanup_old_records(current_time)

        # 检查请求数
        if client_ip in self.request_counts:
            count = self.request_counts[client_ip]["count"]
            if count >= self.max_requests:
                return JSONResponse(
                    status_code=429,
                    content={
                        "success": False,
                        "error": {
                            "code": "RATE_LIMIT_EXCEEDED",
                            "message": "请求过于频繁，请稍后再试"
                        }
                    }
                )
            self.request_counts[client_ip]["count"] += 1
        else:
            self.request_counts[client_ip] = {
                "count": 1,
                "start_time": current_time
            }

        return await call_next(request)

    def _get_client_ip(self, request: Request) -> str:
        """获取客户端 IP"""
        forwarded = request.headers.get("x-forwarded-for")
        if forwarded:
            return forwarded.split(",")[0].strip()
        if request.client:
            return request.client.host
        return "unknown"

    def _cleanup_old_records(self, current_time: float):
        """清理过期记录"""
        expired_ips = []
```

```
        for ip, data in self.request_counts.items():
            if current_time - data["start_time"] > self.window_seconds:
                expired_ips.append(ip)

        for ip in expired_ips:
            del self.request_counts[ip]

# -*- coding: utf-8 -*-
"""
Defect Models
缺陷检测模型
"""
# 引入平台数据库操作模块
# 引入平台 ORM 关系映射模块
from app.core.database import GUID as UUID
from datetime import datetime
import uuid

from app.core.database import Base

class DefectType(Base):
    """缺陷类型表"""
    __tablename__ = "defect_types"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    name = Column(String(100), nullable=False)
    code = Column(String(50), unique=True, nullable=False)
    category = Column(String(50), nullable=True)
    severity = Column(String(20), default="medium")
    description = Column(Text, nullable=True)

    # 检测参数
    detection_params = Column(JSON, nullable=True)

    # 状态
    is_active = Column(Boolean, default=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    defect_results = relationship("DefectResult", back_populates="defect_type")

    def __repr__(self):
        return f"<DefectType {self.name}>"

class DefectSample(Base):
    """缺陷样本表"""
    __tablename__ = "defect_samples"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    sample_no = Column(String(100), unique=True, nullable=False)
    name = Column(String(200), nullable=True)
```

```
    sample_type = Column(String(50), nullable=True)

    # 文件信息
    file_path = Column(String(500), nullable=True)
    file_name = Column(String(200), nullable=True)
    file_size = Column(Integer, nullable=True)
    file_type = Column(String(50), nullable=True)
    image_width = Column(Integer, nullable=True)
    image_height = Column(Integer, nullable=True)

    # 来源
    source = Column(String(50), default="upload")

    # 关联
    device_id = Column(UUID(), ForeignKey("devices.id"), nullable=True)
    device = relationship("Device", back_populates="defect_samples")

    batch_id = Column(UUID(), ForeignKey("batches.id"), nullable=True)
    batch = relationship("Batch", back_populates="defect_samples")

    # 上传者
    uploaded_by = Column(UUID(), ForeignKey("users.id"), nullable=True)
    uploaded_by_user = relationship("User", back_populates="defect_samples")

    # 状态
    status = Column(String(20), default="pending")

    # 元数据
    metadata_json = Column(JSON, nullable=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    defect_results = relationship("DefectResult", back_populates="sample")

    def __repr__(self):
        return f"<DefectSample {self.sample_no}>"

class DefectResult(Base):
    """缺陷检测结果表"""
    __tablename__ = "defect_results"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)

    # 样本关联
    sample_id = Column(UUID(), ForeignKey("defect_samples.id"), nullable=False)
    sample = relationship("DefectSample", back_populates="defect_results")

    # 缺陷类型
    defect_type_id = Column(UUID(), ForeignKey("defect_types.id"),
nullable=True)
    defect_type = relationship("DefectType", back_populates="defect_results")
```

```
    # 检测结果
    has_defect = Column(Boolean, default=False)
    confidence = Column(Float, default=0.0)

    # 位置信息
    bbox = Column(JSON, nullable=True)
    mask = Column(JSON, nullable=True)

    # 分析详情
    analysis_details = Column(JSON, nullable=True)

    # 模型信息
    model_name = Column(String(100), nullable=True)
    model_version = Column(String(50), nullable=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)

    def __repr__(self):
        return f"<DefectResult sample={self.sample_id}
defect={self.has_defect}>"

# -*- coding: utf-8 -*-
"""
Industrial Models
工业资产模型
"""
# 引入平台数据库操作模块
# 引入平台 ORM 关系映射模块
from app.core.database import GUID as UUID
from datetime import datetime
import uuid

from app.core.database import Base

class ProductionLine(Base):
    """生产线表"""
    __tablename__ = "production_lines"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    name = Column(String(100), nullable=False)
    code = Column(String(50), unique=True, nullable=False)
    description = Column(Text, nullable=True)
    location = Column(String(200), nullable=True)
    status = Column(String(20), default="active")

    # 组织关联
    organization_id = Column(UUID(), ForeignKey("organizations.id"),
nullable=True)
    organization = relationship("Organization",
back_populates="production_lines")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    devices = relationship("Device", back_populates="production_line")
    batches = relationship("Batch", back_populates="production_line")

    def __repr__(self):
        return f"<ProductionLine {self.name}>"

class Device(Base):
    """设备表"""
    __tablename__ = "devices"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    name = Column(String(100), nullable=False)
    code = Column(String(50), unique=True, nullable=False)
    device_type = Column(String(50), nullable=True)
    model = Column(String(100), nullable=True)
    manufacturer = Column(String(100), nullable=True)
    serial_number = Column(String(100), nullable=True)
    description = Column(Text, nullable=True)

    # 状态
    status = Column(String(20), default="online")
    health_score = Column(Float, default=100.0)

    # 生产线关联
    production_line_id = Column(UUID(), ForeignKey("production_lines.id"),
nullable=True)
    production_line = relationship("ProductionLine", back_populates="devices")

    # 配置
    config = Column(JSON, nullable=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)
    last_maintenance_at = Column(DateTime, nullable=True)

    # 关联
    defect_samples = relationship("DefectSample", back_populates="device")
    timeseries_datasets = relationship("TimeseriesDataset",
back_populates="device")

    def __repr__(self):
        return f"<Device {self.name}>"

class Batch(Base):
    """生产批次表"""
    __tablename__ = "batches"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    batch_no = Column(String(50), unique=True, nullable=False)
    product_type = Column(String(100), nullable=True)
```

```
    quantity = Column(Integer, default=0)
    status = Column(String(20), default="in_progress")

    # 时间
    start_time = Column(DateTime, nullable=True)
    end_time = Column(DateTime, nullable=True)

    # 生产线关联
    production_line_id = Column(UUID(), ForeignKey("production_lines.id"),
nullable=True)
    production_line = relationship("ProductionLine", back_populates="batches")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    defect_samples = relationship("DefectSample", back_populates="batch")

    def __repr__(self):
        return f"<Batch {self.batch_no}>"

# -*- coding: utf-8 -*-
"""
System Models
系统模型
"""
from enum import Enum
# 引入平台数据库操作模块
# 引入平台 ORM 关系映射模块
from app.core.database import GUID as UUID
from datetime import datetime
import uuid

from app.core.database import Base

class TaskStatus(str, Enum):
    """任务状态枚举"""
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"

class AnalysisJob(Base):
    """分析任务表"""
    __tablename__ = "analysis_jobs"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    name = Column(String(200), nullable=False)
    job_type = Column(String(50), nullable=False)

    # 任务配置
    config = Column(JSON, nullable=True)
```

```
    params = Column(JSON, nullable=True)

    # 状态
    status = Column(String(20), default="pending")
    progress = Column(Float, default=0.0)

    # 结果
    result = Column(JSON, nullable=True)
    error_message = Column(Text, nullable=True)

    # 时间
    started_at = Column(DateTime, nullable=True)
    completed_at = Column(DateTime, nullable=True)

    # 创建者
    created_by = Column(UUID(), ForeignKey("users.id"), nullable=True)
    creator = relationship("User", back_populates="analysis_jobs")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    def __repr__(self):
        return f"<AnalysisJob {self.name} status={self.status}>"

class Notification(Base):
    """通知表"""
    __tablename__ = "notifications"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    title = Column(String(200), nullable=False)
    content = Column(Text, nullable=True)
    notification_type = Column(String(50), default="info")

    # 接收者
    user_id = Column(UUID(), ForeignKey("users.id"), nullable=False)
    user = relationship("User", back_populates="notifications")

    # 状态
    is_read = Column(Boolean, default=False)
    read_at = Column(DateTime, nullable=True)

    # 关联
    related_type = Column(String(50), nullable=True)
    related_id = Column(UUID(), nullable=True)

    # 元数据
    extra_data = Column(JSON, nullable=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)

    def __repr__(self):
        return f"<Notification {self.title}>"
```

```
3233
3234  class AuditLog(Base):
3235      """审计日志表"""
3236      __tablename__ = "audit_logs"
3237
3238      id = Column(UUID(), primary_key=True, default=uuid.uuid4)
3239
3240      # 操作信息
3241      action = Column(String(100), nullable=False)
3242      resource_type = Column(String(50), nullable=True)
3243      resource_id = Column(String(100), nullable=True)
3244
3245      # 操作者
3246      user_id = Column(UUID(), ForeignKey("users.id"), nullable=True)
3247      user = relationship("User", back_populates="audit_logs")
3248
3249      username = Column(String(100), nullable=True)
3250
3251      # 请求信息
3252      ip_address = Column(String(50), nullable=True)
3253      user_agent = Column(String(500), nullable=True)
3254
3255      # 详情
3256      details = Column(JSON, nullable=True)
3257      old_value = Column(JSON, nullable=True)
3258      new_value = Column(JSON, nullable=True)
3259
3260      # 状态
3261      status = Column(String(20), default="success")
3262      error_message = Column(Text, nullable=True)
3263
3264      # 时间戳
3265      created_at = Column(DateTime, default=datetime.utcnow)
3266
3267      def __repr__(self):
3268          return f"<AuditLog {self.action} by {self.username}>"
3269
3270  # -*- coding: utf-8 -*-
3271  """
3272  Timeseries Models
3273  时序分析模型
3274  """
3275  # 引入平台数据库操作模块
3276  # 引入平台 ORM 关系映射模块
3277  from app.core.database import GUID as UUID
3278  from datetime import datetime
3279  import uuid
3280
3281  from app.core.database import Base
3282
3283  class TimeseriesDataset(Base):
3284      """时序数据集表"""
3285      __tablename__ = "timeseries_datasets"
3286
3287      id = Column(UUID(), primary_key=True, default=uuid.uuid4)
```

```
    name = Column(String(100), nullable=False)
    description = Column(Text, nullable=True)

    # 数据源
    source_type = Column(String(50), default="upload")
    file_path = Column(String(500), nullable=True)

    # 设备关联
    device_id = Column(UUID(), ForeignKey("devices.id"), nullable=True)
    device = relationship("Device", back_populates="timeseries_datasets")

    # 数据信息
    data_type = Column(String(50), nullable=True)
    sampling_rate = Column(Float, nullable=True)
    total_points = Column(Integer, default=0)

    # 时间范围
    start_time = Column(DateTime, nullable=True)
    end_time = Column(DateTime, nullable=True)

    # 元数据
    extra_data = Column(JSON, nullable=True)
    columns_config = Column(JSON, nullable=True)

    # 创建者
    created_by = Column(UUID(), ForeignKey("users.id"), nullable=True)
    creator = relationship("User", back_populates="timeseries_datasets")

    # 状态
    status = Column(String(20), default="active")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    data_points = relationship("TimeseriesPoint", back_populates="dataset")
    anomalies = relationship("TimeseriesAnomaly", back_populates="dataset")
    reports = relationship("AnalysisReport", back_populates="dataset")

    def __repr__(self):
        return f"<TimeseriesDataset {self.name}>"

class TimeseriesPoint(Base):
    """时序数据点表"""
    __tablename__ = "timeseries_points"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)

    # 数据集关联
    dataset_id = Column(UUID(), ForeignKey("timeseries_datasets.id"),
nullable=False)
    dataset = relationship("TimeseriesDataset", back_populates="data_points")

```

```
    # 时间戳
    timestamp = Column(DateTime, nullable=False, index=True)

    # 数值
    value = Column(Float, nullable=True)
    values = Column(JSON, nullable=True)

    # 元数据
    extra_data = Column(JSON, nullable=True)
    def __repr__(self):
        return f"<TimeseriesPoint {self.timestamp}>"

class TimeseriesAnomaly(Base):
    """时序异常表"""
    __tablename__ = "timeseries_anomalies"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)

    # 数据集关联
    dataset_id = Column(UUID(), ForeignKey("timeseries_datasets.id"),
nullable=False)
    dataset = relationship("TimeseriesDataset", back_populates="anomalies")

    # 异常信息
    anomaly_type = Column(String(50), nullable=True)
    severity = Column(String(20), default="medium")

    # 时间点
    start_time = Column(DateTime, nullable=False)
    end_time = Column(DateTime, nullable=True)

    # 检测结果
    score = Column(Float, default=0.0)
    description = Column(Text, nullable=True)

    # 详情
    details = Column(JSON, nullable=True)

    # 模型信息
    model_name = Column(String(100), nullable=True)
    model_version = Column(String(50), nullable=True)

    # 状态
    status = Column(String(20), default="detected")
    is_confirmed = Column(Boolean, default=False)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)

    def __repr__(self):
        return f"<TimeseriesAnomaly {self.anomaly_type} at {self.start_time}>"

class AnalysisReport(Base):
    """分析报告表"""
    __tablename__ = "analysis_reports"
```

```
    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    title = Column(String(200), nullable=False)
    report_type = Column(String(50), default="timeseries")

    # 数据集关联
    dataset_id = Column(UUID(), ForeignKey("timeseries_datasets.id"),
nullable=True)
    dataset = relationship("TimeseriesDataset", back_populates="reports")

    # 报告内容
    summary = Column(Text, nullable=True)
    content = Column(JSON, nullable=True)

    # 文件
    file_path = Column(String(500), nullable=True)
    file_format = Column(String(20), default="pdf")

    # 创建者
    created_by = Column(UUID(), ForeignKey("users.id"), nullable=True)
    creator = relationship("User", back_populates="analysis_reports")

    # 状态
    status = Column(String(20), default="draft")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    def __repr__(self):
        return f"<AnalysisReport {self.title}>"
# -*- coding: utf-8 -*-
"""
User Model
用户模型
"""
# 引入平台数据库操作模块
# 引入平台 ORM 关系映射模块
from app.core.database import GUID as UUID
from datetime import datetime
import uuid

from app.core.database import Base

class User(Base):
    """用户表"""
    __tablename__ = "users"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    username = Column(String(50), unique=True, nullable=False, index=True)
    email = Column(String(100), unique=True, nullable=False, index=True)
    hashed_password = Column(String(255), nullable=False)
    full_name = Column(String(100), nullable=True)
    phone = Column(String(20), nullable=True)
```

```
    avatar_url = Column(String(500), nullable=True)

    # 状态
    is_active = Column(Boolean, default=True)
    is_verified = Column(Boolean, default=False)

    # 角色关联
    role_id = Column(UUID(), ForeignKey("roles.id"), nullable=True)
    role = relationship("Role", back_populates="users")

    # 组织/租户
    organization_id = Column(UUID(), ForeignKey("organizations.id"),
nullable=True)
    organization = relationship("Organization", back_populates="users")

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)
    last_login_at = Column(DateTime, nullable=True)

    # 关联
    notifications = relationship("Notification", back_populates="user",
cascade="all, delete-orphan")
    audit_logs = relationship("AuditLog", back_populates="user", cascade="all,
delete-orphan")
    defect_samples = relationship("DefectSample",
back_populates="uploaded_by_user")
    analysis_jobs = relationship("AnalysisJob", back_populates="creator")
    timeseries_datasets = relationship("TimeseriesDataset",
back_populates="creator")
    analysis_reports = relationship("AnalysisReport", back_populates="creator")

    def __repr__(self):
        return f"<User {self.username}>"

class Organization(Base):
    """组织/租户表"""
    __tablename__ = "organizations"

    id = Column(UUID(), primary_key=True, default=uuid.uuid4)
    name = Column(String(100), unique=True, nullable=False)
    code = Column(String(50), unique=True, nullable=False)
    description = Column(Text, nullable=True)
    is_active = Column(Boolean, default=True)

    # 时间戳
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

    # 关联
    users = relationship("User", back_populates="organization")
    production_lines = relationship("ProductionLine",
back_populates="organization")

```

```
    def __repr__(self):
        return f"<Organization {self.name}>"

"""
Industrial AI Platform - Main FastAPI Application
工业智能分析平台 - 主应用入口
"""
# 以下引入平台核心依赖模块（数据库、路由、校验、ORM 等）
import logging
import time

from app.core.config import settings
from app.core.database import engine, Base
from app.api.v1.router import api_router
from app.core.exceptions import AppException
from app.middleware.audit import AuditMiddleware

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """应用生命周期管理"""
    logger.info("Starting Industrial AI Platform...")
    # 创建数据库表（开发环境自动创建）
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    logger.info("Shutting down Industrial AI Platform...")

app = FastAPI(
    title=settings.PROJECT_NAME,
    description="半导体光伏缺陷智能诊断与时序分析系统 - 支持缺陷诊断、时序分析、设备健康管理",
    version="1.0.0",
    openapi_url=f"{settings.API_V1_PREFIX}/openapi.json",
    docs_url=f"{settings.API_V1_PREFIX}/docs",
    redoc_url=f"{settings.API_V1_PREFIX}/redoc",
    lifespan=lifespan
)

# CORS 中间件
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 审计日志中间件
```

```
app.add_middleware(AuditMiddleware)

# 请求计时中间件
@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response

# 全局异常处理
@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "success": False,
            "error_code": exc.error_code,
            "message": exc.message,
            "detail": exc.detail
        }
    )

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    logger.error(f"Unhandled exception: {exc}", exc_info=True)
    return JSONResponse(
        status_code=500,
        content={
            "success": False,
            "error_code": "INTERNAL_ERROR",
            "message": "服务器内部错误",
            "detail": str(exc) if settings.DEBUG else None
        }
    )

# 注册 API 路由
app.include_router(api_router, prefix=settings.API_V1_PREFIX)

# 健康检查
@app.get("/health")
async def health_check():
    return {"status": "healthy", "version": "1.0.0"}

@app.get("/")
async def root():
    return {
        "message": "Welcome to Industrial AI Platform",
        "docs": f"{settings.API_V1_PREFIX}/docs"
    }

# -*- coding: utf-8 -*-
import sqlite3

conn = sqlite3.connect('industrial_ai.db')
```

```
cursor = conn.cursor()

# List all tables
cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
print("Tables:", [row[0] for row in cursor.fetchall()])

# Check defect_samples structure
cursor.execute("PRAGMA table_info(defect_samples)")
print("\ndefect_samples columns:")
for col in cursor.fetchall():
    print(f"  {col[1]}: {col[2]}")

# Check defect_results structure
cursor.execute("PRAGMA table_info(defect_results)")
print("\ndefect_results columns:")
for col in cursor.fetchall():
    print(f"  {col[1]}: {col[2]}")

# Check counts
cursor.execute("SELECT COUNT(*) FROM defect_samples")
print(f"\nDefect samples count: {cursor.fetchone()[0]}")

cursor.execute("SELECT COUNT(*) FROM defect_results")
print(f"Defect results count: {cursor.fetchone()[0]}")

cursor.execute("SELECT COUNT(*) FROM devices")
print(f"Devices count: {cursor.fetchone()[0]}")
```